



SMART CONTRACT AUDIT REPORT

for

DegenX



Prepared By: Xiaomi Huang

PeckShield
December 17, 2024

Document Properties

Client	DegenX
Title	Smart Contract Audit Report
Target	DegenX
Version	1.0
Author	Xuxian Jiang
Auditors	Daisy Cao, Xuxian Jiang
Reviewed by	Xiaomi Huang
Approved by	Xuxian Jiang
Classification	Public

Version Info

Version	Date	Author(s)	Description
1.0	December 17, 2024	Xuxian Jiang	Final Release
1.0-rc	December 5, 2024	Xuxian Jiang	Release Candidate #1

Contact

For more information about this document and its contents, please contact PeckShield Inc.

Name	Xiaomi Huang
Email	contact@peckshield.com

Contents

1	Introduction	4
1.1	About DegenX	4
1.2	About PeckShield	5
1.3	Methodology	5
1.4	Disclaimer	7
2	Findings	9
2.1	Summary	9
2.2	Key Findings	10
3	Detailed Results	11
3.1	Possible Invite Code Stealing in AccessRegistry	11
3.2	Strengthened Input Validated on Function Arguments	12
3.3	Revisited quoteBuyExactTokens() Logic in Gauge	13
3.4	Possibly Earlier Pair Creation Before Completion	14
3.5	Trust Issue of Admin Keys	16
4	Conclusion	18
	References	19

1 | Introduction

Given the opportunity to review the design document and related source code of the `DegenX` protocol, we outline in the report our systematic approach to evaluate potential security issues in the smart contract implementation, expose possible semantic inconsistencies between smart contract code and design document, and provide additional suggestions or recommendations for improvement. Our results show that the given version of smart contracts can be further improved due to the presence of several issues related to either security or performance. This document outlines our audit results.

1.1 About DegenX

`DegenX` is a Meme launchpad that is designed to help creators and developers launch, promote, and monetize meme-themed projects. It aims to lower the barrier for users to participate. The protocol has a specific bonding curve, i.e., $y = k / (q * x_{Total} - x)$, which mimics the behavior of a `UniswapV2 constant-product` AMM with liquidity k . The basic information of audited contracts is as follows:

Table 1.1: Basic Information of DegenX

Item	Description
Name	DegenX
Type	Smart Contract
Language	Solidity
Audit Method	Whitebox
Latest Audit Report	December 17, 2024

In the following, we show the Git repository of reviewed files and the commit hash value used in this audit.

- <https://github.com/degenx-fun/degenx-contracts.git> (a8a832e)

And this is the commit ID after all fixes for the issues found in the audit have been checked in:

- <https://github.com/degenx-fun/degenx-contracts.git> (8e295a9)

1.2 About PeckShield

PeckShield Inc. [9] is a leading blockchain security company with the goal of elevating the security, privacy, and usability of current blockchain ecosystems by offering top-notch, industry-leading services and products (including the service of smart contract auditing). We are reachable at Telegram (<https://t.me/peckshield>), Twitter (<http://twitter.com/peckshield>), or Email (contact@peckshield.com).

Table 1.2: Vulnerability Severity Classification

Impact	High	Medium	Low
	Critical	High	Medium
	High	Medium	Low
Low	Medium	Low	Low
Likelihood			

1.3 Methodology

To standardize the evaluation, we define the following terminology based on OWASP Risk Rating Methodology [8]:

- Likelihood represents how likely a particular vulnerability is to be uncovered and exploited in the wild;
- Impact measures the technical loss and business damage of a successful attack;
- Severity demonstrates the overall criticality of the risk.

Likelihood and impact are categorized into three ratings: *H*, *M* and *L*, i.e., *high*, *medium* and *low* respectively. Severity is determined by likelihood and impact, and can be accordingly classified into four categories, i.e., *Critical*, *High*, *Medium*, *Low* shown in Table 1.2.

To evaluate the risk, we go through a list of check items and each would be labeled with a severity category. For one check item, if our tool or analysis does not identify any issue, the contract is considered safe regarding the check item. For any discovered issue, we might further deploy contracts on our private testnet and run tests to confirm the findings. If necessary, we would

Table 1.3: The Full List of Check Items

Category	Check Item
Basic Coding Bugs	Constructor Mismatch
	Ownership Takeover
	Redundant Fallback Function
	Overflows & Underflows
	Reentrancy
	Money-Giving Bug
	Blackhole
	Unauthorized Self-Destruct
	Revert DoS
	Unchecked External Call
	Gasless Send
	Send Instead Of Transfer
	Costly Loop
	(Unsafe) Use Of Untrusted Libraries
	(Unsafe) Use Of Predictable Variables
	Transaction Ordering Dependence
	Deprecated Uses
Semantic Consistency Checks	Semantic Consistency Checks
Advanced DeFi Scrutiny	Business Logics Review
	Functionality Checks
	Authentication Management
	Access Control & Authorization
	Oracle Security
	Digital Asset Escrow
	Kill-Switch Mechanism
	Operation Trails & Event Generation
	ERC20 Idiosyncrasies Handling
	Frontend-Contract Integration
	Deployment Consistency
	Holistic Risk Management
Additional Recommendations	Avoiding Use of Variadic Byte Array
	Using Fixed Compiler Version
	Making Visibility Level Explicit
	Making Type Inference Explicit
	Adhering To Function Declaration Strictly
	Following Other Best Practices

additionally build a PoC to demonstrate the possibility of exploitation. The concrete list of check items is shown in Table 1.3.

In particular, we perform the audit according to the following procedure:

- Basic Coding Bugs: We first statically analyze given smart contracts with our proprietary static code analyzer for known coding bugs, and then manually verify (reject or confirm) all the issues found by our tool.
- Semantic Consistency Checks: We then manually check the logic of implemented smart contracts and compare with the description in the white paper.
- Advanced DeFi Scrutiny: We further review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs.
- Additional Recommendations: We also provide additional suggestions regarding the coding and development of smart contracts from the perspective of proven programming practices.

To better describe each issue we identified, we categorize the findings with Common Weakness Enumeration (CWE-699) [7], which is a community-developed list of software weakness types to better delineate and organize weaknesses around concepts frequently encountered in software development. Though some categories used in CWE-699 may not be relevant in smart contracts, we use the CWE categories in Table 1.4 to classify our findings. Moreover, in case there is an issue that may affect an active protocol that has been deployed, the public version of this report may omit such issue, but will be amended with full details right after the affected protocol is upgraded with respective fixes.

1.4 Disclaimer

Note that this security audit is not designed to replace functional tests required before any software release, and does not give any warranties on finding all possible security issues of the given smart contract(s) or blockchain software, i.e., the evaluation result does not guarantee the nonexistence of any further findings of security issues. As one audit-based assessment cannot be considered comprehensive, we always recommend proceeding with several independent audits and a public bug bounty program to ensure the security of smart contract(s). Last but not least, this security audit should not be used as investment advice.

Table 1.4: Common Weakness Enumeration (CWE) Classifications Used in This Audit

Category	Summary
Configuration	Weaknesses in this category are typically introduced during the configuration of the software.
Data Processing Issues	Weaknesses in this category are typically found in functionality that processes data.
Numeric Errors	Weaknesses in this category are related to improper calculation or conversion of numbers.
Security Features	Weaknesses in this category are concerned with topics like authentication, access control, confidentiality, cryptography, and privilege management. (Software security is not security software.)
Time and State	Weaknesses in this category are related to the improper management of time and state in an environment that supports simultaneous or near-simultaneous computation by multiple systems, processes, or threads.
Error Conditions, Return Values, Status Codes	Weaknesses in this category include weaknesses that occur if a function does not generate the correct return/status code, or if the application does not handle all possible return/status codes that could be generated by a function.
Resource Management	Weaknesses in this category are related to improper management of system resources.
Behavioral Issues	Weaknesses in this category are related to unexpected behaviors from code that an application uses.
Business Logics	Weaknesses in this category identify some of the underlying problems that commonly allow attackers to manipulate the business logic of an application. Errors in business logic can be devastating to an entire application.
Initialization and Cleanup	Weaknesses in this category occur in behaviors that are used for initialization and breakdown.
Arguments and Parameters	Weaknesses in this category are related to improper use of arguments or parameters within function calls.
Expression Issues	Weaknesses in this category are related to incorrectly written expressions within code.
Coding Practices	Weaknesses in this category are related to coding practices that are deemed unsafe and increase the chances that an exploitable vulnerability will be present in the application. They may not directly introduce a vulnerability, but indicate the product has not been carefully developed or maintained.

2 | Findings

2.1 Summary

Here is a summary of our findings after analyzing the design and implementation of the DegenX protocol. During the first phase of our audit, we study the smart contract source code and run our in-house static code analyzer through the codebase. The purpose here is to statically identify known coding bugs, and then manually verify (reject or confirm) issues reported by our tool. We further manually review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs.

Severity	# of Findings	
Critical	0	
High	0	
Medium	4	
Low	1	
Informational	0	
Total	5	

We have so far identified a list of potential issues: some of them involve subtle corner cases that might not be previously thought of, while others refer to unusual interactions among multiple contracts. For each uncovered issue, we have therefore developed test cases for reasoning, reproduction, and/or verification. After further analysis and internal discussion, we determined a few issues of varying severities need to be brought up and paid more attention to, which are categorized in the above table. More information can be found in the next subsection, and the detailed discussions of each of them are in [Section 3](#).

2.2 Key Findings

Overall, these smart contracts are well-designed and engineered, though the implementation can be improved by resolving the identified issues (shown in Table 2.1), including 4 medium-severity vulnerabilities and 1 low-severity vulnerability.

Table 2.1: Key Audit Findings

ID	Severity	Title	Category	Status
PVE-001	Medium	Possible Invite Code Stealing in Access-Registry	Time and State	Resolved
PVE-002	Low	Strengthened Input Validated on Function Arguments	Coding Practices	Resolved
PVE-003	Medium	Revisited quoteBuyExactTokens() Logic in Gauge	Business Logic	Resolved
PVE-004	Medium	Possibly Earlier Pair Creation Before Completion	Business Logic	Resolved
PVE-005	Medium	Trust Issue Of Admin Keys	Security Features	Mitigated

Beside the identified issues, we emphasize that for any user-facing applications and services, it is always important to develop necessary risk-control mechanisms and make contingency plans, which may need to be exercised before the mainnet deployment. The risk-control mechanisms should kick in at the very moment when the contracts are being deployed on mainnet. Please refer to Section 3 for details.

3 | Detailed Results

3.1 Possible Invite Code Stealing in AccessRegistry

- ID: PVE-001
- Severity: Low
- Likelihood: Low
- Impact: Low
- Target: AccessRegistry
- Category: Time and State [6]
- CWE subcategory: CWE-1126 [1]

Description

The DegenX protocol has a built-in AccessRegistry to allow for the configuration of the user whitelist or blocklist of participation. While reviewing the whitelisted user registration, we notice a possibility that the credential to whitelist a user may be stolen by others.

To elaborate, we show below the implementation of the related routine, i.e., `registerWithInviteCode()`. As the name indicates, it is used to register a whitelisted user with the given invite code. However, the given invite code is validated without having the calling user included. As a result, a malicious user may monitor pending transactions for legitimate `inviteCodeHash` and frontrun with the legitimate one to get him/her registered. As a result, the original user may not be able to register as the stolen `inviteCodeHash` can only be used once.

```
38     function registerWithInviteCode(bytes32 inviteCodeHash, bytes32[] calldata
        merkleProof) external {
39         require(!usedInviteCodes[inviteCodeHash], "AccessRegistry: Invite code already
            used");
40
41         require(MerkleProof.verify(merkleProof, merkleRoot, inviteCodeHash), "
            AccessRegistry: Invalid invite code or proof");
42
43         usedInviteCodes[inviteCodeHash] = true;
44         allowlistedAddresses[msg.sender] = true;
45
46         emit InviteCodeUsed(inviteCodeHash, msg.sender);
47         emit AddressAllowlisted(msg.sender);
```

48

}

Listing 3.1: `AccessRegistry::registerWithInviteCode()`

Recommendation Include the calling user as a part of the `inviteCodeHash` so that the merkle proof credential may not be stolen by others.

Status This issue has been resolved. The team clarifies that the invite code is like where anyone can use any existing invite code, but it won't be in a root specifically.

3.2 Strengthened Input Validated on Function Arguments

- ID: PVE-002
- Severity: Low
- Likelihood: Low
- Impact: Low
- Target: `ConstantProductCurve`
- Category: Business Logic [5]
- CWE subcategory: CWE-837 [3]

Description

As mentioned earlier, the `DegenX` protocol features a unique bonding curve for token price discovery. While examining the specific bonding curve logic, we notice the use of key parameters may be better validated.

For example, the specific bonding curve have a number of key parameters, including `yMin`, `yMax`, `xMax`, and `xTotal`. By design, these input parameters should be validated as follows: `yMax > yMin && yMin > 0 && xMax > 0 && xTotal > xMax`, not current `yMax >= yMin && yMin > 0 && xMax > 0 && xTotal > xMax` (line 66).

```

60     function checkCompatability(
61         uint256 yMin,
62         uint256 yMax,
63         uint256 xMax,
64         uint256 xTotal
65     ) public pure returns (bool) {
66         return yMax >= yMin && yMin > 0 && xMax > 0 && xTotal > xMax;
67     }

```

Listing 3.2: `ConstantProductCurve::checkCompatability()`

Recommendation Improve the above `checkCompatability()` function to ensure the input parameters for the bonding curve are properly validated.

Status This issue has been fixed in the following commit: `8e295a9`.

3.3 Revisited quoteBuyExactTokens() Logic in Gauge

- ID: PVE-003
- Severity: Medium
- Likelihood: Medium
- Impact: Medium
- Target: Gauge
- Category: Business Logic [5]
- CWE subcategory: CWE-837 [3]

Description

The DegenX protocol has a core Gauge contract that allows for real-time discovery of token price. While reviewing the logic for token price discovery, we notice the use of a cost fee and the cost fee deduction needs to be revisited.

To elaborate, we show below the implementation of the related `quoteBuyExactTokens()` function. This function is used to calculate the ETH cost of buying the given amount of tokens, with any fee included. With that, the charged fee needs to be computed as $(\text{rawCost} * \text{feeParameters.buyFee}) / (\text{FRACTION_DENOMINATOR} - \text{feeParameters.buyFee})$, not current $(\text{rawCost} * \text{feeParameters.buyFee}) / \text{FRACTION_DENOMINATOR}$ (line 464).

```

452     function quoteBuyExactTokens(
453         uint256 _amount
454     ) public view returns (uint256, uint256) {
455         uint256 rawCost = curve.evaluateDx(
456             tokenPurchased,
457             tokenPurchased + _amount,
458             curveParameters.yMin,
459             curveParameters.yMax,
460             tokenTarget,
461             totalSupply
462         );
463
464         uint256 fee = (rawCost * feeParameters.buyFee) / FRACTION_DENOMINATOR;
465
466         return (rawCost + fee, fee);
467     }

```

Listing 3.3: Gauge::quoteBuyExactTokens()

Recommendation Revise the above logic to properly compute the fee for the intended token purchase. Note another related routine `quoteSellExactTokens()` shares the same issue.

Status This issue has been fixed in the following commit: 8e295a9.

3.4 Possibly Earlier Pair Creation Before Completion

- ID: PVE-004
- Severity: Medium
- Likelihood: Low
- Impact: High
- Target: Gauge
- Category: Business Logic [5]
- CWE subcategory: CWE-837 [3]

Description

As mentioned earlier, the DegenX protocol has a core Gauge contract that allows for real-time discovery of token price. Moreover, once the bonding process ends, liquidity will be automatically added to the respective pair in UniswapV3. While reviewing the final step of liquidity addition, we notice a possibility that the intended token-weth pair may be created with imbalanced pool before completion.

To elaborate, we show below the implementation of the related `_launchPool()` function. As the name indicates, this routine is invoked when the bonding curve is completed. However, it currently assumes the pair may not be created by others. With that, if a malicious user intentionally creates the pool earlier with an imbalanced state, the added liquidity may result in a loss as it is being added into an imbalanced pool. To fix, there is a need to better protect the liquidity to avoid being manipulated.

```

323     function _launchPool() internal returns (address, uint256) {
324         if (!gaugeActive()) revert Gauge__GaugeClosed();
325         if (!gaugeClosable()) revert Gauge__BondingCurveNotMature();
326
327         // Close the gauge
328         tokenPurchased = 0;
329         tokenTarget = 0;
330         ethPaid = 0;
331
332         // Convert native token to WETH
333         weth.deposit{value: address(this).balance}();
334
335         // Make approvals
336         token.approve(address(poolHelper), type(uint256).max);
337         weth.approve(address(poolHelper), type(uint256).max);
338
339         // Sort tokens
340         address token0;
341         address token1;
342         uint256 amount0;
343         uint256 amount1;
344         if (address(token) < address(weth)) {
345             token0 = address(token);
346             token1 = address(weth);
347             amount0 = token.balanceOf(address(this));

```

```

348         amount1 = weth.balanceOf(address(this));
349     } else {
350         token0 = address(weth);
351         token1 = address(token);
352         amount0 = weth.balanceOf(address(this));
353         amount1 = token.balanceOf(address(this));
354     }
355
356     // Calculate sqrtprice, accounting for rounding in cases where amount1 < amount0
357     uint256 sqrtPriceX96 = (Math.sqrt((amount1 * 1e18) / amount0) *
358         2 ** 96) / 1e9;
359
360     (address _pool, uint256 _tokenId) = poolHelper.createPoolAndMint(
361         address(UNISWAP_V3_NFT_MANAGER),
362         token0,
363         token1,
364         uint160(sqrtPriceX96),
365         lpReceiver,
366         amount0,
367         amount1
368     );
369     pool = _pool;
370
371     // Sweep any dust to the LP receiver
372     if (weth.balanceOf(address(this)) > 0)
373         weth.transfer(lpReceiver, weth.balanceOf(address(this)));
374     if (token.balanceOf(address(this)) > 0)
375         token.transfer(lpReceiver, token.balanceOf(address(this)));
376
377     emit PoolLaunched(address(token), _pool, _tokenId);
378
379     return (_pool, _tokenId);
380 }

```

Listing 3.4: Gauge::_launchPool()

Recommendation Revise the above logic to properly complete the bonding curve and protect the liquidity.

Status The issue was originally confirmed and later resolved in the following commit: 10250aa.

3.5 Trust Issue of Admin Keys

- ID: PVE-005
- Severity: Medium
- Likelihood: Medium
- Impact: Medium
- Target: Multiple Contracts
- Category: Security Features [4]
- CWE subcategory: CWE-287 [2]

Description

In the DegenX protocol, there is a privileged `owner` account that plays a critical role in governing and regulating the system-wide operations (e.g., configure parameters, pause/unpause gauges, and execute privileged operations). It also has the privilege to control or govern the flow of assets managed by this protocol. Our analysis shows that the privileged account needs to be scrutinized. In the following, we examine the privileged account and the related privileged accesses in current contracts.

```

783     function setTokenImplementation(address _tokenImplementation) external onlyOwner
784         {...}
785     function setGaugeImplementation(address _gaugeImplementation) external onlyOwner
786         {...}
787     function setAccessRegistry(address _accessRegistry) external onlyOwner {...}
788     ...
789     function setCurve(address _curve) external onlyOwner {...}
790     ...
791     function setTreasury(address _treasury) external onlyOwner {...}
792     ...
793     function setLpReceiver(address _lpReceiver) external onlyOwner {...}
794     ...
795     function setCreationCost(uint64 _creationCost) external onlyOwner {...}
796     ...
797     function setBoosterFraction(uint64 _boosterFraction) external onlyOwner {...}
798     ...
799     function setGaugeFees(Gauge.FeeParameters calldata _gaugeFees) external onlyOwner
        {...}

```

Listing 3.5: Example Privileged Functions in Launchpad

Note that if the privileged `owner` account is a plain EOA account, this may be worrisome and pose counter-party risk to the exchange users. A multi-sig account could greatly alleviate this concern, though it is still far from perfect. Specifically, a better approach is to eliminate the administration key concern by transferring the role to a community-governed DAO. In the meantime, a timelock-based mechanism can also be considered as mitigation.

Recommendation Promptly transfer the privileged account to the intended DAO-like governance contract. All changed to privileged operations may need to be mediated with necessary timelocks. Eventually, activate the normal on-chain community-based governance life-cycle and ensure the intended trustless nature and high-quality distributed governance.

Status This issue has been mitigated as the team makes use of a multisig to act as the privileged owner.



4 | Conclusion

In this audit, we have analyzed the design and implementation of the `DegenX` protocol, which is a `MemeLaunchpad` designed to help creators and developers launch, promote, and monetize meme-themed projects. It aims to lower the barrier for users to participate. The protocol has a specific bonding curve, i.e., $y = k / (q * x_{Total} - x)$, which mimics the behavior of a `UniswapV2` `constant-product` AMM with liquidity k . The current code base is well structured and neatly organized. Those identified issues are promptly confirmed and addressed.

Meanwhile, we need to emphasize that `Solidity`-based smart contracts as a whole are still in an early, but exciting stage of development. To improve this report, we greatly appreciate any constructive feedbacks or suggestions, on our methodology, audit findings, or potential gaps in scope/coverage.



References

- [1] MITRE. CWE-1126: Declaration of Variable with Unnecessarily Wide Scope. <https://cwe.mitre.org/data/definitions/1126.html>.
- [2] MITRE. CWE-287: Improper Authentication. <https://cwe.mitre.org/data/definitions/287.html>.
- [3] MITRE. CWE-837: Improper Enforcement of a Single, Unique Action. <https://cwe.mitre.org/data/definitions/837.html>.
- [4] MITRE. CWE CATEGORY: 7PK - Security Features. <https://cwe.mitre.org/data/definitions/254.html>.
- [5] MITRE. CWE CATEGORY: Business Logic Errors. <https://cwe.mitre.org/data/definitions/840.html>.
- [6] MITRE. CWE CATEGORY: Concurrency. <https://cwe.mitre.org/data/definitions/557.html>.
- [7] MITRE. CWE VIEW: Development Concepts. <https://cwe.mitre.org/data/definitions/699.html>.
- [8] OWASP. Risk Rating Methodology. https://www.owasp.org/index.php/OWASP_Risk_Rating_Methodology.
- [9] PeckShield. PeckShield Inc. <https://www.peckshield.com>.